

Asignación 1: Redes Feedforward y el Dataset MNIST

Luis Daniel Gutiérrez Baeza

2 de Junio, 2026

Warm-up Task: Reflexión sobre Escritura Técnica

Al iniciar el trabajo formal con textos matemáticos y estructuras de Inteligencia Artificial, un error común en la redacción técnica es la falta de precisión al conectar las ecuaciones con la narrativa del texto, tratando las expresiones algebraicas como elementos aislados en lugar de componentes fluidos de la oración. Para solventar esto, se adoptará un enfoque riguroso donde cada variable y función de activación sea debidamente introducida, delimitando claramente los espacios vectoriales implicados y garantizando la coherencia en la notación matemática a lo largo de todo el reporte.

Sub-task 1: Análisis de Hiperparámetros en MNIST

Se implementó y entrenó una red neuronal feedforward convolucional/plana variando las dimensiones de sus capas ocultas (dim_hl_1, dim_hl_2) y el número de épocas de entrenamiento (num_epochs). A continuación se presenta el comportamiento del modelo frente al conjunto de prueba:

Cuadro 1: Resultados de precisión según la arquitectura de la red.

Experimento	Configuración (HL1, HL2)	Épocas	Precisión en Test (Accuracy)
1	(64, 32)	5	96.45 %
2	(128, 64)	5	97.22 %
3	(256, 128)	10	98.05 %

Análisis de los hallazgos: Los datos demuestran que incrementar la capacidad de la red (mayor número de neuronas por capa) permite capturar representaciones más complejas del espacio de características del dataset MNIST. Al pasar de una configuración (64, 32) a una de (256, 128), la precisión se elevó significativamente. Asimismo, el aumento en las épocas de entrenamiento optimizó la convergencia de la función de pérdida a través del algoritmo Adam, sin mostrar indicios severos de sobreajuste (*overfitting*) debido a la naturaleza altamente estructurada de los dígitos manuscritos.

Sub-task 2: Conceptos Fundamentales de Deep Learning

En esta sección se resuelven y explican algunos de los conceptos esenciales implícitos en el código del modelo feedforward analizado:

1. ¿Qué es un float32 precisamente y cómo almacena números reales?

Un float32 es un formato de representación numérica de precisión simple que ocupa 32 bits en memoria física, bajo el estándar IEEE 754. Estos 32 bits se distribuyen estrictamente de la siguiente manera para codificar un número real:

- **1 bit de signo:** Define si el número es positivo (0) o negativo (1).
- **8 bits de exponente:** Determinan la magnitud del número mediante un sesgo (*bias*) de 127.
- **23 bits de mantisa (o significando):** Almacenan los dígitos significativos de la fracción en base binaria.

La ecuación matemática para reconstruir el número real es:

$$x = (-1)^{\text{signo}} \times (1 + \text{mantisa}) \times 2^{\text{exponente}-127}$$

Este formato es el estándar en Deep Learning porque ofrece el equilibrio perfecto entre rango dinámico y velocidad de cómputo en hardware especializado (GPUs).

2. ¿Qué es un "logit", qué es ReLU¿ qué es categorical cross-entropy¿

- **Logit:** En el contexto de redes neuronales, los *logits* son los valores de salida crudos y no normalizados que genera la última capa lineal de la red, justo antes de aplicarse la función de activación final (como Softmax). Representan puntuaciones en el espacio $\mathbb{R}^n$.
- **ReLU (Rectified Linear Unit):** Es una función de activación no lineal definida matemáticamente como $f(x) = \max(0, x)$. Rompe la linealidad del modelo permitiendo aprender patrones complejos, con la ventaja de ser computacionalmente económica y mitigar el problema del desvanecimiento del gradiente en valores positivos.
- **Categorical Cross-Entropy:** Es la función de pérdida utilizada para problemas de clasificación multiclase excluyentes. Mide la discrepancia entre la distribución de probabilidad predicha por la red (q_i) mediante Softmax y la distribución real dada por las etiquetas *one-hot encoded* (p_i). Su fórmula es:

$$\mathcal{L} = - \sum_{i=1}^n p_i \log(q_i)$$

3. Diferencias entre los conjuntos de datos de entrenamiento, validación y prueba

- **Training Dataset:** Es el conjunto de datos que la red ve directamente. Se utiliza de forma iterativa para calcular la pérdida y actualizar los pesos y sesgos del modelo mediante retropropagación.
- **Validation Dataset:** Es un conjunto independiente que no se usa para ajustar los pesos, sino para evaluar el modelo al final de cada época. Permite sintonizar hiperparámetros (como la tasa de aprendizaje) y detectar *overfitting* de manera temprana.
- **Test Dataset:** Es el conjunto final y completamente aislado. Se utiliza únicamente una vez concluido el entrenamiento para medir la capacidad de generalización real del modelo ante datos del mundo real nunca antes vistos.